

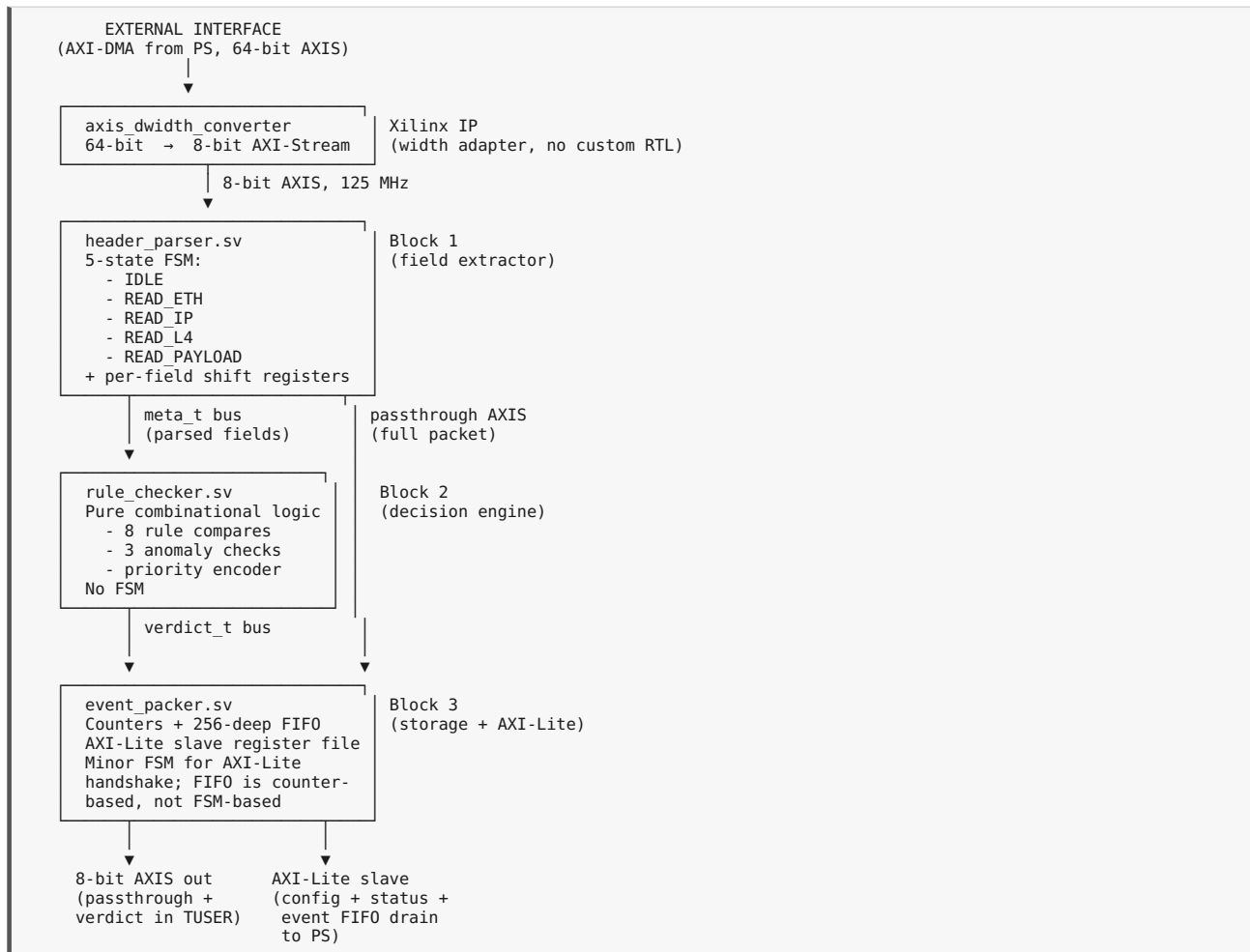
Phase 1 — System Reference & FSM Appendix

Project: Line-Rate Network Packet Inspection Engine on PYNQ-Z2 **Scope:** All ports, all signal connections, all FSMs, all dataflow paths.

This document is meant to be skim-readable during interviews and complete enough to serve as appendix material for an eventual Phase 2 paper.

Part 1 — System Architecture Overview

High-level block diagram



Why this architecture

- **Byte-oriented internal datapath (8-bit):** Field extraction is trivial — one byte per cycle, each field accumulates in a shift register. Mirrors well-understood streaming parser patterns (NASDAQ ITCH, MIPI parsers). 125 MHz × 8 bits = 1 Gbps, exactly matching the 1 Gbps Ethernet target.
- **AXI-Stream width converter at the front:** Xilinx's `axis_dwidth_converter` adapts the 64-bit DMA stream to 8-bit transparently. No custom RTL is required for the width adaptation; the converter handles buffering and beat alignment.
- **Three custom RTL blocks with clean interfaces:** Each block has one job: parser extracts, rule_checker decides, event_packer stores. Each is independently unit-testable.
- **Stateless rule_checker:** All eight rules and all three anomaly checks evaluated in a single clock cycle, in parallel. This is the FPGA's structural advantage over a CPU — no sequential rule traversal.
- **Decoupled producer/consumer via FIFO:** The PL can generate one event per packet at line rate (worst case); the PS polls the FIFO at its own (much slower) pace. The 256-deep FIFO absorbs bursts and reports overflow via a sticky status bit.

- **Verdict rides in-band on TUSER:** The passthrough output stream carries both the original packet AND the verdict bits in the TUSER sideband. Downstream consumers see verdict alongside the packet without a separate channel.

Part 2 — Module Port Specifications

Block 1: header_parser.sv

Purpose: Walk the byte stream, capture Ethernet/IPv4/TCP/UDP header fields, emit a structured metadata record at end-of-packet.

Internal state: - 5-state FSM: IDLE, READ_ETH, READ_IP, READ_L4, READ_PAYLOAD - byte_count [10:0] — position within current packet (max ~1518 bytes for jumbo-free Ethernet) - frame_length [15:0] — total received byte count, captured at TLAST - Field shift registers: dst_mac[47:0], src_mac[47:0], eth_type[15:0], ip_version_ihl[7:0], ip_total_length[15:0], ip_protocol[7:0], ip_src[31:0], ip_dst[31:0], l4_src_port[15:0], l4_dst_port[15:0], tcp_flags[7:0]

Port	Direction	Width	Description
aclk	input	1	Clock (125 MHz target)
aresetn	input	1	Synchronous active-low reset
s_axis_tdata	input	8	Byte from upstream width converter
s_axis_tvalid	input	1	Source has valid data
s_axis_tready	output	1	Sink can accept (always 1 in v1)
s_axis_tlast	input	1	End-of-packet marker
m_axis_tdata	output	8	Passthrough byte
m_axis_tvalid	output	1	Passthrough valid
m_axis_tready	input	1	Downstream readiness
m_axis_tlast	output	1	Passthrough TLAST
m_axis_tuser_payload_valid	output	1	HIGH on beats containing payload bytes
meta	output	struct (~280 bits)	Packed metadata bus (see struct below)
meta_valid	output	1	One-cycle pulse coincident with TLAST

Metadata struct (packet_meta_t):

```
typedef struct packed {
    logic [47:0] eth_dst_mac;
    logic [47:0] eth_src_mac;
    logic [15:0] eth_type;
    logic [3:0] ip_version;
    logic [3:0] ip_ihl;
    logic [15:0] ip_total_length;
    logic [7:0] ip_protocol;
    logic [31:0] ip_src;
    logic [31:0] ip_dst;
    logic [15:0] l4_src_port;
    logic [15:0] l4_dst_port;
    logic [7:0] tcp_flags;
    logic [15:0] frame_length;
    logic [15:0] payload_offset;
    logic is_ipv4;
    logic is_tcp;
    logic is_udp;
    logic parser_error;
} packet_meta_t;
```

Block 2: rule_checker.sv

Purpose: Compare incoming metadata against 8 software-configured rules and 3 hard-coded anomaly checks. Emit a single-cycle verdict.

Internal state: - None at the comparison level (purely combinational from meta to verdict) - Verdict is registered

for one cycle to ease downstream timing

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Synchronous active-low reset
meta	input	struct	Packed metadata from parser
meta_valid	input	1	Metadata strobe
rule_table	input	8×128 bits	Rule entries (driven from event_packer)
verdict	output	struct (~20 bits)	Decision bus
verdict_valid	output	1	One-cycle strobe (registered, ≈1 cycle after meta_valid)

Rule entry (rule_entry_t):

```
typedef struct packed {
    logic [31:0] src_ip;
    logic [31:0] dst_ip;
    logic [15:0] src_port;      // 0 = any
    logic [15:0] dst_port;      // 0 = any
    logic [7:0]  src_prefix_len; // CIDR mask, 0-32
    logic [7:0]  dst_prefix_len; // CIDR mask, 0-32
    logic [7:0]  protocol;      // 0 = any
    logic [2:0]  action;         // 0 = accept, 1 = drop, 2 = flag-only
    logic [3:0]  reserved;
    logic        enable;
} rule_entry_t;
```

Verdict struct (verdict_t):

```
typedef struct packed {
    logic [7:0] rule_hit_mask; // bit N = rule N matched
    logic [2:0] rule_id;       // lowest-index matching rule (priority-encoded)
    logic [2:0] anomaly_bits;  // {ip_len_mismatch, tcp_syn_fin, bad_ihl}
    logic        any_hit;      // OR of hit_mask | anomaly_bits
    logic        drop;         // 1 if any matching rule.action == drop
} verdict_t;
```

Anomaly definitions:

Bit	Anomaly	Condition
[0]	bad_ihl	is_ipv4 && (ip_ihl < 5)
[1]	tcp_syn_fin	is_tcp && tcp_flags[0] && tcp_flags[1] (SYN ^ FIN)
[2]	ip_len_mismatch	is_ipv4 && (ip_total_length != frame_length - 14)

Block 3: event_packer.sv

Purpose: Maintain protocol-level counters; capture verdicts into a 256-deep event FIFO when any_hit; expose all counters, the rule table, and the FIFO drain port through an AXI-Lite slave port. Forward the passthrough AXI-Stream to the top-level output with verdict bits inserted into TUSER.

Internal state: - 256-deep × 160-bit event FIFO (built from BRAM) - PACKET_COUNT[31:0], DROP_COUNT[31:0], ANOMALY_COUNT[31:0], RULE_HIT_N[31:0] × 8 - STATUS bits: fifo_empty, fifo_full, fifo_overflow (sticky), parser_error_sticky - CONTROL bits: enable, clear_counters (self-clearing pulse) - Minor AXI-Lite slave FSM (idle/write/read) for register interface - Rule table register file: 8 × rule_entry_t (driven out to rule_checker) - Free-running 32-bit timestamp counter (increments every cycle)

Port	Direction	Width	Description
<code>aclk</code> , <code>aresetn</code>	input	1	Clock, reset
<code>s_axis_*</code>	input	(AXI-S)	Stream passthrough from parser, with <code>m_axis_tuser_payload_valid</code>
<code>verdict</code>	input	struct	Verdict from rule_checker
<code>verdict_valid</code>	input	1	Verdict strobe
<code>meta</code>	input	struct	Shadow copy of metadata (for event capture)
<code>m_axis_*</code>	output	(AXI-S)	Final stream output with <code>TUSER[15:0]</code> carrying verdict
<code>rule_table</code>	output	8×128 bits	Configured rules (drives rule_checker)
<code>s_axi_lite_*</code>	input/output	(AXI-Lite)	Standard 32-bit AXI-Lite slave interface
<code>irq</code>	output	1	Asserted when <code>EVENT_FIFO_DEPTH > 0</code> (optional Phase 1 use)

Part 3 — Cross-Module Connection Map

Wiring diagram for `packet_inspector_top.sv`.

Stream path

Signal: `s_axis_tdata` (**external → width converter**) - Source: top-level input from AXI-DMA - Destination: `axis_dwidth_converter.S_AXIS.TDATA[63:0]` - Purpose: 64-bit AXI-Stream packet ingress from PS

Signal: `conv_to_parser.tdata` (**converter → parser**) - Source: `axis_dwidth_converter.M_AXIS.TDATA[7:0]` - Destination: `header_parser.s_axis_tdata` - Purpose: 8-bit serialized byte stream

Signal: `parser_to_packer.tdata` (**parser → event_packer**) - Source: `header_parser.m_axis_tdata` - Destination: `event_packer.s_axis_tdata` - Purpose: One-cycle-delayed passthrough byte

Signal: `m_axis_tdata` (**event_packer → external**) - Source: `event_packer.m_axis_tdata` - Destination: top-level output - Purpose: Final byte stream with verdict in `TUSER`

Metadata and verdict paths

Signal: `meta` (**parser → rule_checker, also → event_packer**) - Source: `header_parser.meta` (packed struct, ~280 bits) - Destinations: `rule_checker.meta`, `event_packer.meta` - Purpose: Parsed header fields. `event_packer` needs it for event capture; `rule_checker` needs it for comparison.

Signal: `meta_valid` (**parser → rule_checker, → event_packer**) - Source: `header_parser.meta_valid` - Destinations: `rule_checker.meta_valid`, `event_packer.meta_valid` (for `PACKET_COUNT` increment) - Purpose: One-cycle strobe at end-of-packet

Signal: `verdict` (**rule_checker → event_packer**) - Source: `rule_checker.verdict` (registered) - Destination: `event_packer.verdict` - Purpose: Decision bus

Signal: `verdict_valid` (**rule_checker → event_packer**) - Source: `rule_checker.verdict_valid` - Destination: `event_packer.verdict_valid` - Purpose: One-cycle strobe; arrives one cycle after `meta_valid`

Signal: `rule_table` (**event_packer → rule_checker**) - Source: `event_packer.rule_table` (8 × `rule_entry_t`) - Destination: `rule_checker.rule_table` - Purpose: PS-configured rules read by the matching logic

AXI-Lite path

Signal: `s_axi_lite_*` (**external → event_packer**) - Source: top-level AXI-Lite slave port - Destination: `event_packer.s_axi_lite_*` - Purpose: PS reads counters, writes rules, drains event FIFO

TUSER assembly

`m_axis_tuser[15:0]` is assembled in `event_packer` as: - `TUSER[7:0] = verdict.rule_hit_mask` - `TUSER[10:8] = verdict.anomaly_bits` - `TUSER[14:11] = verdict.rule_id` (3 bits + 1 reserved) - `TUSER[15] = verdict.drop`

Part 4 — FSM Deep Dive

What is an FSM?

A Finite State Machine is a circuit that remembers what it's currently doing (its state) and decides what to do next based on (a) its current state and (b) the current inputs.

Hardware FSMs have: - A finite set of named states (`IDLE` , `READ_ETH` , etc.) - A state register — flip-flops storing the current state - Transition logic — combinational logic computing next state from current state + inputs - Output logic — combinational logic computing outputs from current state (and optionally inputs)

Why use FSMs?

When a circuit's behavior depends on what happened before, not just what's on the inputs right now. Examples in this project: - "Is this byte part of the Ethernet header, IP header, or payload?" — depends on how many bytes have been seen so far. - "Should the parser still be capturing IP fields, or has it moved on to L4?" — depends on the FSM's current state.

Pure combinational logic can't carry that memory; FSMs add it.

How an FSM works — per-cycle view

1. At the clock edge: the state register captures whatever `next_state` was during the previous cycle. The state register now holds the new state.
2. During the cycle (combinational):
 - The transition logic looks at the current state and the inputs, and computes `next_state` .
 - The output logic looks at the current state (and inputs, for Mealy FSMs) and computes outputs.
3. At the next clock edge: `next_state` becomes the new current state. Repeat.

Two-process FSM design (what we use)

```
// Process 1: State register (sequential)
always_ff @(posedge aclk) begin
    if (!aresetn)
        state <= IDLE;
    else
        state <= next_state;
end

// Process 2: Next-state logic (combinational)
always_comb begin
    next_state = state; // default: stay
    case (state)
        IDLE:          if (s_axis_tvalid)          next_state = READ_ETH;
        READ_ETH:      if (byte_count == 13)       next_state = is_ipv4 ? READ_IP : READ_PAYLOAD;
        READ_IP:       if (byte_count == 33)       next_state = (is_tcp || is_udp) ? READ_L4 : READ_PAYLOAD;
        READ_L4:       if (byte_count == l4_header_end) next_state = READ_PAYLOAD;
        READ_PAYLOAD:  if (s_axis_tlast)          next_state = IDLE;
    endcase
end
```

Outputs are typically assigned with simple `assign` statements (Moore-style, depending only on state), or with combinational case statements (Mealy-style, depending on state and inputs).

Mealy vs Moore

- **Moore:** Outputs depend ONLY on the current state. Output is stable for the entire cycle the state is active. Easier to reason about.
- **Mealy:** Outputs depend on state AND inputs. Can save a state at the cost of glitch susceptibility.

This project uses primarily Moore outputs (state-driven), with one Mealy-style output: `m_axis_tuser_payload_valid` is `(state == READ_PAYLOAD) && s_axis_tvalid` , which is Mealy because it depends on the input strobe.

State encoding

We use binary encoding via `typedef enum` . For our 5-state parser FSM, 3 bits is enough. For our 2-3-state event_packer mini-FSMs, 2 bits suffices.

Synchronous vs asynchronous reset

We use synchronous active-low reset (`if (!aresetn)` inside `always_ff @(posedge aclk)`). Standard practice on Xilinx FPGAs; simpler than asynchronous and works fine for clock-synchronous designs.

Part 5 — Our Specific FSMs

header_parser FSM (5 states)

States:

- **IDLE** — No packet in flight. Waiting for `s_axis_tvalid` to assert, indicating the first byte of a new packet has arrived.
- **READ_ETH** — Reading the 14-byte Ethernet header. Captures `dst_mac` (bytes 0-5), `src_mac` (bytes 6-11), and `eth_type` (bytes 12-13).
- **READ_IP** — Reading the 20-byte IPv4 header (bytes 14-33 of packet). Captures `ip_version_ihl`, `ip_total_length`, `ip_protocol`, `ip_src`, `ip_dst`.
- **READ_L4** — Reading the L4 (TCP/UDP) header. Captures `l4_src_port`, `l4_dst_port`, and (for TCP) `tcp_flags`.
- **READ_PAYLOAD** — Streaming payload bytes until `TLAST`. No field capture; asserts `m_axis_tuser_payload_valid` for downstream Phase 2 use.

Transitions:

From	To	Condition
IDLE	READ_ETH	<code>s_axis_tvalid && s_axis_tready</code>
READ_ETH	READ_IP	<code>byte_count == 13 && eth_type == 0x0800</code>
READ_ETH	READ_PAYLOAD	<code>byte_count == 13 && eth_type != 0x0800</code>
READ_IP	READ_L4	<code>byte_count == 33 && (ip_protocol == 6 ip_protocol == 17)</code>
READ_IP	READ_PAYLOAD	<code>byte_count == 33 && other protocols</code>
READ_L4	READ_PAYLOAD	<code>byte_count == (33 + l4_header_length)</code>
any	IDLE	<code>s_axis_tlast</code> (with <code>meta_valid</code> pulse)

Outputs: - `m_axis_tdata`, `m_axis_tvalid`, `m_axis_tlast`: 1-cycle passthrough of inputs (registered). - `m_axis_tuser_payload_valid`: `(state == READ_PAYLOAD) && s_axis_tvalid`. - `meta_valid`: 1-cycle pulse coincident with `s_axis_tlast`, regardless of which state was active when `TLAST` arrived.

rule_checker — no FSM, combinational only

The rule checker deliberately avoids an FSM. It's a pure combinational compare on `meta_valid` cycles, with the verdict registered on the next clock edge for downstream timing.

Logic structure: - 8 parallel rule comparators (each: `src_ip` masked compare + `dst_ip` masked compare + port match + protocol match, all AND'd with `rule.enable`) - 3 parallel anomaly detectors (one per anomaly bit) - A priority encoder turning `rule_hit_mask` into `rule_id` (lowest-index hit) - A drop-flag OR-tree: any matching rule with `action == drop` sets `verdict.drop`

This is the FPGA's core advantage made explicit: 8 rules + 3 anomalies = 11 comparison operations, all happening in one cycle.

event_packer mini-FSMs

The event_packer has two small auxiliary FSMs and one main counter-based structure:

AXI-Lite slave FSM (3 states): `IDLE`, `WRITE`, `READ`. Standard AXI-Lite handshake; one transaction in flight at a time. Drives the rule table writes, register reads, and FIFO drain.

Event FIFO control: Counter-based, NOT FSM. A write-pointer increments when a verdict arrives with `any_hit`; a read-pointer increments when the PS writes to `EVENT_FIFO_POP`. The `EVENT_FIFO_DEPTH` register is the difference.

Counter increments: Combinational decode on each `verdict_valid` cycle. `PACKET_COUNT++` always; `RULE_HIT_N++` for each set bit in `hit_mask`; `DROP_COUNT++` if `drop`; `ANOMALY_COUNT++` if any anomaly bit set.

Part 6 — Cycle-by-Cycle Dataflow

Walkthrough of a single TCP SYN packet flowing through the pipeline. Example packet: 60 bytes, source

192.168.1.5:54321, destination 8.8.8.8:53 (a TCP SYN to Google's DNS on port 53). After the 64→8 width converter, the packet arrives at the parser as 60 sequential bytes over 60 cycles.

Assume rule 0 is configured as “flag any traffic to destination port 53.” All other rules are disabled.

For brevity, the table below shows key transition cycles, not all 60.

Cycle	byte_count	s_axis_tdata	parser state	meta_valid	verdict_valid	What happens
1	0	0x00	IDLE → READ_ETH	0	0	First byte of dst_mac; transition to READ_ETH
2-6	1-5	dst_mac bytes	READ_ETH	0	0	dst_mac shift register accumulates
7-12	6-11	src_mac bytes	READ_ETH	0	0	src_mac shift register accumulates
13-14	12-13	0x08 0x00	READ_ETH	0	0	EtherType captured = 0x0800 (IPv4)
15	14	0x45	READ_ETH → READ_IP	0	0	Version + IHL captured (4, 5 = normal); transition to READ_IP
16-19	15-18	IP fields	READ_IP	0	0	TOS, Total Length captured
20-21	19-20	ID bytes	READ_IP	0	0	Identification
22-23	21-22	flags+fragoff	READ_IP	0	0	Flags + Fragment Offset
24	23	0x40	READ_IP	0	0	TTL = 64
25	24	0x06	READ_IP	0	0	Protocol = 6 (TCP)
26-27	25-26	checksum	READ_IP	0	0	Header Checksum
28-31	27-30	src IP bytes	READ_IP	0	0	ip_src shift register: 0xC0A80105
32-34	31-33	dst IP bytes	READ_IP	0	0	ip_dst shift register accumulating
35	34	last dst_ip byte	READ_IP → READ_L4	0	0	ip_dst = 0x08080808; protocol was TCP → READ_L4
36-37	35-36	src_port bytes	READ_L4	0	0	l4_src_port capture: 0xD431 = 54321
38-39	37-38	dst_port bytes	READ_L4	0	0	l4_dst_port capture: 0x0035 = 53
40-46	39-45	seq/ack/offset	READ_L4	0	0	Skipped fields
47	46	0x02	READ_L4	0	0	TCP flags = 0x02 (SYN only)
48-53	47-52	rest of TCP	READ_L4 → READ_PAYLOAD	0	0	After byte 53 (TCP done at byte 53 = 34+20-1)
54-59	53-58	payload (none in SYN)	READ_PAYLOAD	0	0	TCP SYN has no payload in this minimal frame
60	59	last byte, TLAST=1	READ_PAYLOAD → IDLE	1	0	meta_valid pulses; metadata struct stable
61	—	—	IDLE	0	1	rule_checker registered verdict valid: hit_mask=0b00000001, rule_id=0, anomaly_bits=0, drop=0
62	—	—	IDLE	0	0	event_packer increments PACKET_COUNT, RULE_HIT_0; writes one entry to event FIFO

Key observations

1. **Latency through pipeline:** From `s_axis_tlast` at the parser input to the event being captured in the FIFO is 2 cycles (parser registers `meta_valid` on cycle 60; rule_checker registers `verdict_valid` on cycle 61; event_packer captures on cycle 62). Throughput is limited only by the byte arrival rate, not by pipeline depth.
 2. **No backpressure between custom blocks:** The parser, rule_checker, and event_packer don't push back on each other in v1. The whole pipeline stalls naturally if the upstream width converter de-asserts `TVALID`.
 3. **Per-byte field capture is the simplest possible thing:** Each cycle, exactly one shift register updates based on `byte_count`. A case statement on `byte_count` selects which field. No multi-byte alignment logic, no field straddling beat boundaries.
 4. **Idle cycles propagate cleanly:** When `TVALID` goes low between packets, `byte_count` freezes, the FSM stays in `READ_PAYLOAD`, and nothing changes downstream. The next `TLAST`, whenever it eventually comes, still triggers `meta_valid` correctly.
 5. **Unknown protocols are silently dropped:** If `EtherType != 0x0800`, the parser bypasses `READ_IP` and `READ_L4`, going directly to `READ_PAYLOAD`. The `meta` struct emerges with `is_ipv4 = 0`; the rule_checker observes that and doesn't match any rule that requires IPv4 fields. No anomaly fires either. The packet flows through with a "neutral" verdict.
-

Part 7 — Interview Talking Points

"Walk me through the design"

"It's a 3-stage AXI-Stream pipeline at 125 MHz. The first stage is a header parser — a 5-state FSM that walks each byte of the packet, identifies Ethernet/IPv4/TCP/UDP fields by their byte offsets, and emits a structured metadata record at end-of-packet. The second stage is a rule checker — pure combinational logic that compares the metadata against 8 parallel rules and 3 anomaly detectors in a single clock cycle. The third stage is an event packer — it maintains counters, captures hits into a 256-deep FIFO, and exposes everything to the PS over AXI-Lite. The whole pipeline is 8-bit byte-oriented internally; a Xilinx width converter adapts the 64-bit DMA stream to 8-bit at the front end. Single byte per cycle, no FIFOs between custom stages, deterministic latency."

"Why use an FSM in the parser but not the rule checker?"

"The parser's behavior depends on what happened before — whether we've seen the Ethernet header yet, whether the EtherType was IPv4, whether the protocol was TCP. That's textbook FSM territory. The rule checker, by contrast, evaluates a function purely of the current packet's metadata — no history needed. So the parser is sequential and stateful; the rule checker is combinational. Using an FSM where you don't need one would just add latency and verification surface."

"Why is the rule checker combinational instead of pipelined?"

"With 8 rules and 3 anomaly checks running in parallel on a small metadata bus, the comparison logic fits comfortably within one 125 MHz clock period. Pipelining would add latency without throughput benefit since the rule checker is already meeting timing. If we scaled to 64 or 128 rules, or to deeper masking like LPM with priority resolution, pipelining would start to make sense."

"How do you handle a packet whose EtherType isn't IPv4?"

"The parser bypasses the IP and L4 reading states and goes straight to `READ_PAYLOAD`. The metadata struct emerges with `is_ipv4 = 0`. Downstream the rule checker sees this and rejects all IP-based rules (because IP fields are zero/garbage). No anomaly fires because anomalies are conditioned on `is_ipv4`. The packet passes through unflagged. This is intentional: the parser doesn't make policy decisions, and 'I don't recognize this' is a perfectly valid output."

"What's the latency from packet end to event capture?"

"Two cycles. The parser pulses `meta_valid` on the same cycle as `s_axis_tlast`. The rule checker registers the verdict, so `verdict_valid` pulses one cycle later. The event packer captures into the FIFO on the cycle after that. Total: `TLAST` → 2 cycles → event visible to the PS via AXI-Lite read."

"Why byte-oriented internally instead of 64-bit native?"

"For Phase 1, the simplicity of byte-oriented field extraction far outweighs the bandwidth headroom of going wider. Fields live at byte boundaries and accumulate one byte at a time into shift registers — there's no multi-byte alignment logic, no logic to handle fields straddling beat boundaries. The Xilinx width converter adapting 64-bit-to-8-bit is free RTL from the IP catalog. At 125 MHz × 8 bits = 1 Gbps, we exactly match the line rate. Going wider would have been a premature optimization."

“How would you handle backpressure if the downstream sink stalled?”

“Currently the design assumes the downstream sink is always ready. Adding backpressure means each stage needs to propagate `tready` upstream and stall its own internal state when downstream isn’t ready. The parser’s `byte_count` should freeze, the rule_checker should hold its verdict until accepted, the event_packer should hold the FIFO write. The width converter already supports backpressure on both ports, so it’s just a matter of plumbing `tready` correctly through the custom modules. This is a v2 enhancement.”

“Why is the event FIFO 256 entries deep?”

“It’s a balance. Too small and we lose events during a burst of matching traffic; too large and we waste BRAM. 256 entries at 160 bits is one BRAM-36 block, which is the smallest unit Xilinx synthesis naturally allocates. At 1 Gbps with minimum-size 64-byte packets, that’s ~2 million packets per second worst case. If every packet flagged, the PS has ~125 µs to drain before the FIFO fills — which is plenty for a polling loop running at 1 kHz. Overflow is reported as a sticky status bit so we know if it ever happens.”

“What would you do in Phase 2?”

“Phase 2 adds payload pattern matching. A new module, `pattern_matcher`, watches the parser’s `m_axis_tuser_payload_valid` and looks for one or two configurable byte patterns in the payload using a shift-register windowed compare. Matches feed into the existing event FIFO as a new event type. The Phase 1 design left this clean by exposing the payload-valid sideband, reserving event-type code 2, and reserving address space `0x80–0xFF` in the AXI-Lite map for pattern configuration. So Phase 2 is purely additive.”

Part 8 — File Manifest

```
rtl/
├── packet_inspector_top.sv      Top-level wrapper with all three blocks
│                               + axis_dwidth_converter instantiation
├── packet_inspector_pkg.sv     Shared SystemVerilog package
│                               (packet_meta_t, rule_entry_t, verdict_t)
├── header_parser.sv           Block 1 – 5-state FSM, field shift registers
├── rule_checker.sv            Block 2 – combinational compare,
│                               priority encoder
└── event_packer.sv            Block 3 – counters, FIFO, AXI-Lite slave

ip/
├── (Xilinx IP – instantiated from block design, not custom RTL)
├── axis_dwidth_converter      64-bit → 8-bit AXIS adaptation
├── axi_dma                    PS to PL packet ingress
├── axis_data_fifo             Optional elasticity buffer
└── axi_interconnect           AXI-Lite control plane routing

tb/
├── tb_header_parser.sv        Unit testbench, directed Ethernet frames
├── tb_rule_checker.sv         Unit testbench, metadata-matrix
├── tb_event_packer.sv         Unit testbench, AXI-Lite + FIFO
└── tb_top_pcap.sv             System-level testbench, replays PCAP file

golden_model/
├── inspector_reference.py      Python reference implementation
└── pcap_to_hex.py             Converts PCAP to testbench hex stimulus

sw/
├── packet_inspector_overlay.py PYNQ overlay class
├── live_demo.ipynb             Jupyter notebook with live capture + dashboard
└── pcap_replay.py              Headless PCAP-replay benchmark

docs/
├── packet_inspector_proposal.md IEEE-style project proposal
├── packet_inspector_spec.md     Module-level interface spec
├── packet_inspector_interview_prep.md Concept-level interview prep Q&A
└── packet_inspector_system_reference.md This document
```

Part 9 — Glossary

- **AXI-Stream:** ARM AMBA’s standard streaming-data interface protocol. Uses `TVALID` / `TREADY` handshake plus optional sideband (`TKEEP` , `TLAST` , `TUSER`). Natural fit for packet data.
- **AXI-Lite:** ARM AMBA’s standard memory-mapped register interface. 32-bit address/data, one transaction in flight. Used for the PS to configure the PL and read status.
- **TUSER:** Optional sideband signal on AXI-Stream. Width is user-defined. Carries per-beat or per-packet metadata. We use it to carry the verdict alongside the packet.
- **TLAST:** AXI-Stream sideband bit indicating end-of-packet. Asserts on the last beat of a packet.

- **MAC address:** 6-byte (48-bit) hardware address identifying a NIC on a local network segment. Only meaningful within one broadcast domain.
- **EtherType:** 2-byte field at the end of the Ethernet header indicating what protocol is inside. `0x0800` = IPv4.
- **IHL:** Internet Header Length. 4-bit field giving the IPv4 header size in 32-bit words. Normal value is 5 (= 20 bytes).
- **Protocol field:** 1-byte field in the IPv4 header indicating what protocol is inside. 6 = TCP, 17 = UDP, 1 = ICMP.
- **TCP flags:** 6 control bits in byte 13 of the TCP header. FIN, SYN, RST, PSH, ACK, URG.
- **5-tuple:** Canonical flow identifier — `(src_ip, dst_ip, src_port, dst_port, protocol)`. Uniquely identifies a connection at L3/L4.
- **CIDR / prefix-length mask:** A way to specify a range of IP addresses using a prefix length (0-32 for IPv4). `192.168.1.0/24` means “match the first 24 bits, ignore the last 8.”
- **IDS / IPS:** Intrusion Detection / Prevention System. An IDS observes and reports; an IPS observes and blocks. This project is an IDS in Phase 1.
- **FIFO:** First-In-First-Out buffer. Holds events between hardware generation and software consumption.
- **Network byte order:** Big-endian. The most significant byte is sent first on the wire.
- **One-hot encoding:** Representation where exactly one bit is high at any time. We don’t use one-hot in the parser FSM (binary-encoded), but the rule hit-mask is essentially one-hot after the priority encode.
- **Synthesizable RTL:** SystemVerilog code that the synthesis tool can convert into physical logic gates. Excludes constructs like `$display`, `initial`, dynamic arrays.
- **Elaboration:** Vivado’s first compile pass. Parses code, checks syntax, builds the module hierarchy.
- **Synthesis:** The second pass. Converts elaborated RTL into a gate-level netlist mapped to FPGA primitives.
- **Place-and-route:** Final implementation pass. Decides which physical FPGA location each gate goes in and how to wire them. Determines actual `Fmax` and resource usage.
- **Fmax:** Maximum clock frequency at which the design meets all timing constraints. Reported by Vivado after place-and-route.
- **BRAM:** Block RAM. Dedicated SRAM blocks in the FPGA fabric. 36 Kb each on Zynq-7000. Used here for the event FIFO.
- **LUT:** Look-Up Table. The basic combinational primitive in an FPGA. 6-input on Zynq-7000.

End of Phase 1 System Reference. Save this in `docs/packet_inspector_system_reference.md` and update it as you add the system testbench, synthesis results, and on-board bring-up notes.